

[Harvin AI/](#)

Improving store count detection accuracy

Shared by Rahul

This is a copy of a chat between Claude and Rahul. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

HarvinAI_TechScanner_Technical_Documentation.md

975 lines

md

Here is the Technical architecture for your reference. My team is currently implementing this but facing some challenges related to accuracy of the store count detection of the physical stores a d2c brand has . I want you to go through it and help me make it accurate and also suggest improvements. Take a deep breath and work on this step-by-step.

Mar 6

Rahul, this is one of the highest-value but trickiest data points in your entire model. Store count is a leading indicator for omnichannel expansion — exactly the kind of signal that Raj at CleverTap and Priya at her agency would pay for. But it's also where most intelligence platforms fail because there's no single clean source of truth.

Let me break down the problem and give you a concrete architecture for solving it.

Why Store Count Detection Is Hard

The core challenge is that there's no canonical database of "stores owned by Brand X."

Every source has noise:

Google Maps/Places API — Lists franchise locations, authorized resellers, and multi-brand retailers alongside brand-owned stores. A search for "Lenskart" returns both company stores and partner opticians. You'll over-count by 30-60%.

Brand store locator pages — Most authoritative, but implementations vary wildly. Some are static HTML lists, some are JS-rendered (React/Angular SPAs that load via API calls), some use third-party widgets (Storemapper, Storepoint, Bullseye). Many are outdated — stores close but don't get removed.

Press releases / news — Good for detecting new openings ("Brand X opens 50th store in Mumbai") but useless for current total count. Also noisy with franchise vs company-owned distinctions.

Job postings — Useful as a signal ("hiring store manager in Pune" = likely new store) but not for counting.

Recommended Architecture: Multi-Source Triangulation Pipeline

The right approach isn't picking the "best" source — it's building a pipeline that triangulates across sources, assigns confidence scores, and flags anomalies for manual review.

Here's the system I'd recommend:

Source Priority Stack

Tier 1 (Primary — highest signal): Brand's own store locator page. This is your ground truth attempt. Most D2C brands have one at /stores, /store-locator, /our-stores, or /find-us.

Tier 2 (Validation): Google Places API — used to validate Tier 1 counts, not as primary source. Cross-reference to catch stores the brand forgot to list, or to flag when Google shows far more results (indicating reseller contamination).

Tier 3 (Signal enrichment): News/PR monitoring + job postings — used for change detection ("Brand opened 5 new stores this quarter") rather than absolute counts.

Tier 1: Store Locator Scraping Engine

This is where you should invest most of your engineering effort. Here's the detailed approach:

Step 1 — Discover the store locator URL

Most D2C brands follow predictable URL patterns. Build a crawler that checks these paths on each brand's domain:

/stores

/store-locator

/our-stores

/find-us

/find-a-store

/locations

/retail-stores

/offline-stores

/store-finder

/visit-us

Also scan the site's navigation (header/footer links) for anchor text containing "store," "visit," "find us," "locations." Your Puppeteer infrastructure from TechScanner is perfect for this — you already have the stealth browser setup.

Step 2 — Classify the store locator type

This is critical because extraction logic differs completely by type. You'll encounter roughly five patterns:

Pattern A — Static HTML list. Stores are rendered server-side as HTML elements (divs, list items). Example: many Shopify brands using a simple page. Detection: check if store data is present in initial HTML without JS execution. Extraction: DOM parsing with CSS selectors.

Pattern B — JS-rendered SPA. Page loads empty, then fetches store data via XHR/fetch and renders client-side. Example: brands on headless CMS or custom React apps. Detection: HTML is mostly empty on initial load, but populated after Puppeteer's `networkidle2`.

Extraction: intercept the XHR response (this is the gold — you get structured JSON directly).

Pattern C — Third-party widget. Brand embeds Storemapper, Storepoint, Stockist, Bullseye, or similar. Detection: look for known widget script URLs or iframes. Extraction: call the widget's underlying API directly (most have predictable endpoints).

Pattern D — Google Maps embed with pins. Brand embeds a Google Map with custom pins. Detection: iframe src containing google.com/maps/embed or Maps JavaScript API usage. Extraction: harder — may need to parse KML/GeoJSON data or fall back to Tier 2.

Pattern E — PDF/image-based. Some brands (especially Indian D2C) just have a PDF or image listing stores. Detection: check for PDF links or image-heavy pages with no structured data. Extraction: flag for manual processing.

Step 3 — Extract and structure store data

For each store locator, extract: store name/label, city, state/region, pin code/zip, latitude/longitude (if available), store type (if labeled — flagship, kiosk, outlet, partner). The XHR interception approach (Pattern B) is by far the most reliable. Here's why it's powerful and how to implement it:

javascript

```
// Inside your Puppeteer scrape, intercept API responses
```

```
const storeData = [];
```

```
page.on('response', async (response) => {
  const url = response.url();
  // Common store locator API patterns
  const storeApiPatterns = [
    /store/i, /location/i, /branch/i,
    /outlet/i, /retail/i, /find.*store/i,
    /storemapper/i, /storepoint/i, /stockist/i
  ];

  if (storeApiPatterns.some(p => p.test(url))) {
    try {
      const contentType = response.headers()['content-type'] || '';
      if (contentType.includes('json')) {
        const json = await response.json();
        storeData.push({ url, data: json });
      }
    } catch (e) { /* not JSON, skip */ }
  }
});
```

```
await page.goto(storeLocatorUrl, { waitUntil: 'networkidle2' });
```

```
// Also trigger "show all stores" if there's a search
```

```
// Many locators require a search input before showing results
```

```
// Try: zoom out to max, or search for the country name
```

```
...
```

The key insight: **many store locators lazy-load stores by geography.** If you just visit the page, you might see 0 stores until you search. You need to either trigger a broad search (search for the country name, or set the map bounds to the entire country) or find the "view all" API endpoint.

****Step 4 — Deduplicate and normalize****

Raw store data will have duplicates (same store listed under slightly different names) and noise (warehouses, offices, pop-up counters). Apply these rules:

Normalize addresses: strip whitespace, standardize abbreviations (St → Street, Rd → Road). Deduplicate on (city + pin code) or (lat/long within 100m radius). Filter out entries containing keywords like "warehouse," "office," "headquarters," "coming soon." Flag entries with "partner," "dealer," "authorized" — these are likely not brand-owned.

Tier 2: Google Places API Validation

Use the Google Places API ****not**** as your primary count, but as a cross-reference:
'''

```
// Search: "Brand Name" + store
// Restrict to specific country/region
// Filter by place type: "store" or "point_of_interest"
Compare the Places API count against your Tier 1 count:
```

- Tier 1 $\approx$ Tier 2 (within 20%): High confidence. Use Tier 1 count.
- Tier 1 $\gg$ Tier 2: Brand's locator may include partners/dealers. Flag for review.
- Tier 1 $\ll$ Tier 2: Brand's locator is likely outdated or geo-restricted. Investigate.
- Tier 2 $\gg$ Tier 1 by 3x+: Almost certainly reseller contamination in Google results. Trust Tier 1.

Important cost consideration: Google Places API charges per request. At 500K brands, even one search per brand per month = significant cost. Prioritize: only run Places API validation on brands where you have active watchlist subscribers, or where Tier 1 failed.

Tier 3: Change Detection via News & Jobs

This isn't for absolute counts — it's for detecting changes to trigger a re-scrape:
Monitor for signals like: "[Brand] opens [Nth] store in [City]," "[Brand] plans to open [N] stores by [date]," job postings for "Store Manager" or "Retail Associate" in new cities.
When a change signal fires, re-run the Tier 1 scraper for that brand and update the count.
This is also your source for the "Omnichannel Expansion" signal in the Market Intelligence dashboard.

Confidence Scoring Model

Every store count should have an attached confidence score. Here's a simple but effective model:

Factor	Score
--------	-------

Tier 1 (store locator) successfully scraped with structured data	+40
XHR/API intercept (got clean JSON)	+20
Tier 2 (Google Places) validates within 20%	+15
Count verified manually in last 90 days	+15
Multiple sources agree	+10
Store locator page last updated < 6 months	+5
Deductions:	
Factor	Score
Only Google Places data, no store locator found	-30
Store locator requires geo-search (may have incomplete coverage)	-15
Count changed by >50% since last check (suspicious)	-20
Brand known to have franchise model (count may include franchisees)	-10
Thresholds: 70+ = show with confidence, 40-69 = show with "approximate" label, below 40 = show as "data unavailable" rather than showing a bad number. This is critical — showing an inaccurate store count is worse than showing no count at all. If Vikram the VC sees "12 stores" when the real number is 45, your credibility is destroyed.	

Specific Improvements to Your Current System

Based on the TechScanner architecture, here's what I'd leverage and what needs to change:
 Reuse from TechScanner: Your three-tier fetching strategy (Puppeteer → Axios → cURL) is directly applicable. The Puppeteer + Stealth setup is essential for store locators that are SPAs. The browser singleton pattern saves you from Chrome cold-start costs.

What needs to be different:

First, TechScanner visits the homepage only. Store detection needs to discover and visit a separate URL (the store locator page). Build a "store locator discovery" step that runs before scraping.

Second, TechScanner does pattern matching against HTML. Store detection needs to intercept network requests and parse structured data from API responses. This is a fundamentally different extraction model — you're not regex-matching, you're capturing and parsing JSON payloads.

Third, TechScanner is stateless (scan → return → forget). Store detection needs to be stateful — you need to compare today's count against last week's count to detect openings/closings. Add a `store_counts` collection in MongoDB:

```
javascript
{
  domain: "lenskart.com",
  current_count: 1200,
  previous_count: 1150,
  confidence_score: 82,
  source: "store_locator_api", // or "google_places", "manual"
  store_locator_url: "https://www.lenskart.com/stores",
  locator_type: "xhr_api",    // static_html, spa, widget, maps_embed
  last_scraped: ISODate("2026-03-01"),
  last_changed: ISODate("2026-02-15"),
  stores: [                  // optional: full store list for detail pages
    { city: "Mumbai", area: "Andheri West", pincode: "400058" },
    // ...
  ]
}
```

Fourth, add a known-widget registry. Many D2C brands use the same store locator SaaS products. Build dedicated parsers for the top 5-6:

- Storemapper — API at <https://www.storemapper.co/api/users/{id}/stores.json>
- Storepoint — API at <https://api.storepoint.co/v1/{id}/locations>
- Stockist — API at <https://stockist.co/api/v1/{id}/locations>
- Bullseye — Various embed patterns
- Shopify native store locator — Usually at `/apps/store-locator` with predictable API

If you detect one of these widgets during the TechScanner scan (you already can — just add detection patterns), you can skip the generic Puppeteer scrape entirely and hit the widget API directly. This is faster, more reliable, and cheaper.

Risks & Trade-offs

Data freshness vs. cost: Scraping store locators for 500K brands monthly is expensive (Puppeteer compute time, Google Places API costs). Recommendation: scrape the full database quarterly, but re-scrape weekly for brands on active watchlists. This aligns compute cost with paying user value.

Franchise ambiguity: For brands like Lenskart or FabIndia that have a mix of company-owned and franchise stores, you may not be able to distinguish. Be transparent — show "1,200+ retail locations" rather than "1,200 stores" and add a note where franchise models are detected. Don't pretend precision you don't have.

International coverage: Store locators are often country-specific. A brand may have separate locators for India, US, and UAE. Your scraper needs to handle multi-region discovery.

Recommended Implementation Order

Week 1-2: Build store locator URL discovery (check common paths + nav scraping). Classify locator types for your top 100 brands manually to understand the distribution.

Week 3-4: Build extractors for the two most common patterns (likely XHR/API intercept and static HTML parsing). Add known-widget parsers for Storemapper and Storepoint.

Week 5-6: Add Google Places API validation layer. Build confidence scoring. Set up the store_counts MongoDB collection with change tracking.

Week 7-8: Run against your full brand database. Manual QA on a sample of 200 brands. Tune confidence thresholds based on accuracy.

This is a 2-month workstream for one backend engineer, but it produces one of your most defensible data assets. No competitor in your space (BuiltWith, StoreLeads, even SimilarWeb) does store count tracking at this level for D2C brands specifically. That's your moat.

Want me to go deeper on any specific part — the XHR interception implementation, the widget parser registry, or the confidence scoring model?

To run code, enable code execution and file creation in Settings > Capabilities.

Yes

Mar 7

Let me go deep on all three. I'll make this implementation-ready so your backend engineer can start building.

1. XHR Interception Engine — Full Implementation

The core idea: instead of parsing messy HTML, intercept the actual API calls that store locator pages make to fetch store data. You get clean, structured JSON — no regex, no DOM traversal, no fragile CSS selectors.

The Problem With HTML Parsing

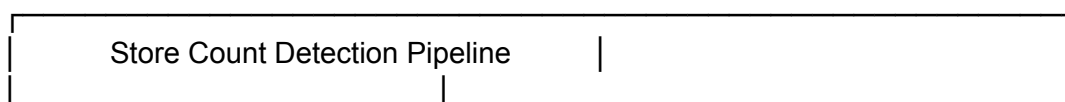
Most modern store locators are SPAs. When Puppeteer visits the page, the initial HTML is a near-empty shell. The actual store data loads via an XHR/fetch call to an API endpoint, then gets rendered client-side. If you try to parse the rendered DOM, you're fighting against:

Different HTML structures per brand. Dynamic class names (CSS modules, Tailwind).

Lazy-loaded map markers that only appear after user interaction. Pagination that hides most stores behind "load more" buttons.

But the underlying API response? That's almost always clean JSON with a consistent structure per brand. And here's the key insight — you only need to reverse-engineer the API once per store locator type, not once per brand. Brands on Shopify tend to use the same handful of store locator apps. Brands on custom stacks tend to use the same SaaS widgets. The long tail of truly custom implementations is smaller than you'd think.

Architecture



- | | |
|---|--|
| 1. Navigate to store locator URL | |
| 2. Register network interceptors | |
| 3. Wait for page load (networkidle2) | |
| 4. If no store data captured: | |
| → Try trigger strategies (search, zoom out) | |
| 5. If still no data: | |
| → Fall back to DOM parsing | |
| 6. Parse captured responses | |
| 7. Deduplicate + normalize | |
| 8. Return structured store list | |

Core Implementation

Here's the full interceptor engine. This would live as a new module alongside your existing detect.js:

javascript

// server/store-detector.js

```
const STORE_API_PATTERNS = [
  // Generic patterns that match most store locator APIs
  /\stores/i,
  /\locations/i,
  /\branches/i,
  /\outlets/i,
  /\store-locator/i,
  /\find-store/i,
  /\retail/i,
  /\apiV.*store/i,
  /\apiV.*location/i,
  /\graphql/i, // Many modern stores use GraphQL

  // Known widget API endpoints
  /storemapper\.co\/api/i,
  /storepoint\.co\/api/i,
  /stockist\.co\/api/i,
  /cdn\.shopify\.com\/s\/files.*store/i,
  /bullseyelocations\.com/i,
  /sweetiq\.com/i,
  /yext\.com.*locations/i,
  /uberall\.com/i,
  /chatmeter\.com/i,
];

// Patterns to EXCLUDE (these fire on store locator pages
// but don't contain store data)
const EXCLUDE_PATTERNS = [
  /google-analytics/i,
```

```

/googletagmanager/i,
/facebook\.com/i,
/hotjar/i,
/maps\.googleapis\.com\/maps\/api\/js/i, // Maps JS loader, not data
/fonts\.googleapis/i,
\.css(?:|$)/i,
\.png|\.jpg|\.gif|\.svg|\.webp/i,
];

```

```

async function interceptStoreAPIs(page, storeLocatorUrl) {
  const capturedResponses = [];

  // Register interceptor BEFORE navigation
  page.on('response', async (response) => {
    const url = response.url();
    const status = response.status();

    // Skip non-200 responses and excluded patterns
    if (status !== 200) return;
    if (EXCLUDE_PATTERNS.some(p => p.test(url))) return;

    // Check if URL matches store API patterns
    const isStoreAPI = STORE_API_PATTERNS.some(p => p.test(url));
    if (!isStoreAPI) return;

    try {
      const contentType = response.headers()['content-type'] || '';

      if (contentType.includes('json') || contentType.includes('text')) {
        const text = await response.text();

        // Try JSON parse
        let parsed = null;
        try {
          parsed = JSON.parse(text);
        } catch {
          // Not JSON — might be JSONP or HTML, skip
          return;
        }

        // Score this response: does it look like store data?
        const score = scoreStoreResponse(parsed);

        if (score > 0) {
          capturedResponses.push({
            url,
            data: parsed,
            score,

```

```

        size: text.length,
    });
    }
}
} catch (e) {
    // Response body already consumed or network error — skip
}
});

// Navigate to the store locator page
await page.goto(storeLocatorUrl, {
    waitUntil: 'networkidle2',
    timeout: 30000
});

// Wait a bit more for lazy-loaded API calls
await page.waitForTimeout(2000);

// If we didn't capture anything, try trigger strategies
if (capturedResponses.length === 0) {
    await tryTriggerStrategies(page);
    await page.waitForTimeout(3000);
}

return capturedResponses;
}

```

The Response Scoring Function

This is the secret sauce — not every JSON response on a store locator page contains store data. You need a heuristic that distinguishes "this is a list of store locations" from "this is analytics config" or "this is CMS content."

javascript

```

function scoreStoreResponse(data) {
    let score = 0;

    // Flatten: if data is wrapped in a common envelope, unwrap it
    const payload = unwrapEnvelope(data);

    // Must be an array or contain an array
    const arrays = findArraysInObject(payload);
    if (arrays.length === 0) return 0;

    for (const { path, arr } of arrays) {
        // Skip tiny arrays (< 2 items) and huge arrays (> 10000, likely not stores)
        if (arr.length < 2 || arr.length > 10000) continue;

        // Check first 3 items for store-like properties

```

```

const sample = arr.slice(0, 3);
let arrayScore = 0;

for (const item of sample) {
  if (typeof item !== 'object' || item === null) continue;

  const keys = Object.keys(item).map(k => k.toLowerCase());
  const values = Object.values(item).map(v =>
    typeof v === 'string' ? v.toLowerCase() : v
  );

  // High-signal keys (strong indicators of store data)
  const highSignalKeys = [
    'latitude', 'longitude', 'lat', 'lng', 'long', 'lon',
    'address', 'street', 'city', 'state', 'zipcode', 'zip',
    'pincode', 'postal_code', 'postalcode',
    'store_name', 'storename', 'location_name',
    'phone', 'store_phone', 'store_id', 'store_code',
    'opening_hours', 'hours', 'timings',
  ];

  // Medium-signal keys
  const medSignalKeys = [
    'name', 'title', 'country', 'region', 'area',
    'coordinates', 'geo', 'position', 'location',
    'district', 'landmark',
  ];

  const highMatches = highSignalKeys.filter(k =>
    keys.some(key => key.includes(k))
  ).length;

  const medMatches = medSignalKeys.filter(k =>
    keys.some(key => key.includes(k))
  ).length;

  // Check for coordinate-like values
  const hasCoords = values.some(v => {
    if (typeof v !== 'number') return false;
    // Lat range: -90 to 90, Lng range: -180 to 180
    return (v > -90 && v < 90) || (v > -180 && v < 180);
  });

  if (highMatches >= 2) arrayScore += 30;
  else if (highMatches >= 1) arrayScore += 15;

  if (medMatches >= 2) arrayScore += 10;
  if (hasCoords) arrayScore += 20;
}

```

```

    }

    // Bonus for array size in realistic store range
    if (arr.length >= 3 && arr.length <= 5000) arrayScore += 10;

    score = Math.max(score, arrayScore);
  }

  return score;
}

function unwrapEnvelope(data) {
  // Common API response wrappers
  if (data?.data) return data.data;
  if (data?.results) return data.results;
  if (data?.response) return data.response;
  if (data?.stores) return data.stores;
  if (data?.locations) return data.locations;
  if (data?.features) return data.features; // GeoJSON format
  if (data?.body) return data.body;
  return data;
}

function findArraysInObject(obj, path = "", depth = 0) {
  if (depth > 4) return []; // Don't go too deep
  const results = [];

  if (Array.isArray(obj)) {
    results.push({ path, arr: obj });
  } else if (obj && typeof obj === 'object') {
    for (const [key, value] of Object.entries(obj)) {
      results.push(
        ...findArraysInObject(value, `${path}.${key}`, depth + 1)
      );
    }
  }
}

return results;
}

```

Trigger Strategies

Many store locators show nothing until the user interacts. Here are the strategies to force data loading:

javascript

```

async function tryTriggerStrategies(page) {
  // Strategy 1: Click "View All Stores" or "Show All" buttons
  const viewAllSelectors = [

```

```

'button:has-text("View All")',
'button:has-text("Show All")',
'button:has-text("All Stores")',
'button:has-text("See All")',
'a:has-text("View All")',
'a:has-text("All Locations")',
'[class*="view-all"]',
'[class*="show-all"]',
'[data-action="show-all"]',
];

for (const selector of viewAllSelectors) {
  try {
    const el = await page.$(selector);
    if (el) {
      await el.click();
      await page.waitForTimeout(2000);
      return 'view_all_clicked';
    }
  } catch { /* selector not found, try next */ }
}

```

// Strategy 2: Search for a broad term

```

const searchSelectors = [
  'input[placeholder*="search"]',
  'input[placeholder*="city"]',
  'input[placeholder*="location"]',
  'input[placeholder*="pincode"]',
  'input[placeholder*="zip"]',
  'input[type="search"]',
  '#store-search',
  '.store-search input',
  '[class*="locator"] input',
];

```

```

for (const selector of searchSelectors) {
  try {
    const input = await page.$(selector);
    if (input) {
      // Type a broad search term — country name works well
      await input.type('India', { delay: 50 });

      // Try pressing Enter or clicking a search button
      await page.keyboard.press('Enter');
      await page.waitForTimeout(2000);
      return 'search_triggered';
    }
  } catch { continue; }
}

```

```
}
```

```
// Strategy 3: If there's a Google Map, zoom out to max
```

```
try {  
  const mapContainer = await page.$('[class*="map"]', #map, .gm-style');  
  if (mapContainer) {  
    // Scroll to zoom out (negative deltaY = zoom out)  
    const box = await mapContainer.boundingBox();  
    if (box) {  
      await page.mouse.move(  
        box.x + box.width / 2,  
        box.y + box.height / 2  
      );  
      for (let i = 0; i < 10; i++) {  
        await page.mouse.wheel({ deltaY: -500 });  
        await page.waitForTimeout(200);  
      }  
      await page.waitForTimeout(2000);  
      return 'map_zoomed_out';  
    }  
  }  
} catch { /* no map found */ }
```

```
// Strategy 4: Check for pagination / "Load More"
```

```
const loadMoreSelectors = [  
  'button:has-text("Load More")',  
  'button:has-text("Show More")',  
  '[class*="load-more"]',  
  '[class*="pagination"] a:last-child',  
];
```

```
for (const selector of loadMoreSelectors) {  
  try {  
    let clicked = 0;  
    while (clicked < 20) { // Safety limit  
      const el = await page.$(selector);  
      if (!el) break;  
      await el.click();  
      await page.waitForTimeout(1000);  
      clicked++;  
    }  
    if (clicked > 0) return `load_more_clicked_${clicked}`;  
  } catch { continue; }  
}
```

```
return 'no_trigger_worked';  
}
```

DOM Fallback Parser

When XHR interception captures nothing (static HTML pages, or obscure implementations), fall back to DOM parsing:

javascript

```
async function fallbackDOMParsing(page) {
  return await page.evaluate(() => {
    const stores = [];

    // Strategy A: Look for structured data (JSON-LD)
    const jsonLdScripts = document.querySelectorAll(
      'script[type="application/ld+json"]'
    );
    for (const script of jsonLdScripts) {
      try {
        const data = JSON.parse(script.textContent);
        // Look for LocalBusiness or Store types
        const items = Array.isArray(data) ? data : [data];
        for (const item of items) {
          if (item['@type']?.match(/Store|LocalBusiness|Place/i)) {
            stores.push({
              name: item.name,
              address: item.address?.streetAddress,
              city: item.address?.addressLocality,
              state: item.address?.addressRegion,
              zip: item.address?.postalCode,
              lat: item.geo?.latitude,
              lng: item.geo?.longitude,
              source: 'json_ld'
            });
          }
        }
      } catch { /* invalid JSON-LD */ }
    }

    if (stores.length > 0) return stores;

    // Strategy B: Find repeating DOM patterns that look like store cards
    // Look for containers with multiple similar children
    // that contain address-like content
    const candidates = document.querySelectorAll(
      '[class*="store"], [class*="location"], [class*="branch"], ' +
      '[class*="outlet"], [data-store], [data-location]'
    );

    for (const el of candidates) {
      const text = el.textContent.trim();
      // Simple heuristic: if element contains a phone number
```

```

// or pincode pattern, it's likely a store card
const hasPhone = /(\\+91|0\\d{2,4})[\\s-]?\\d{6,10}/.test(text);
const hasPincode = \\b\\d{6}\\b/.test(text); // Indian pincode
const hasZip = \\b\\d{5}(-\\d{4})?\\b/.test(text); // US zip

if (hasPhone || hasPincode || hasZip) {
  stores.push({
    raw_text: text.substring(0, 500),
    source: 'dom_parsing'
  });
}
}

return stores;
});
}

```

2. Widget Parser Registry

This is your highest-ROI investment after the XHR interceptor. A surprisingly large percentage of D2C brands use the same 6-8 store locator SaaS tools. Build dedicated parsers for each and you cover a huge chunk of your database with high accuracy.

How to Detect Which Widget a Brand Uses

You already have the infrastructure for this — your TechScanner detection engine. Add these entries to your TECH array in detect.js:

javascript

// Add to curated TECH entries in detect.js

```

{
  name: 'Storemapper',
  category: 'Store Locator Widget',
  color: '#4A90D9',
  detect: {
    scripts: [/storemapper\\.co/i, /storemapper\\.com/i],
    html: [/storemapper-embed/i, /class="storemapper"/i],
  }
},
{
  name: 'Storepoint',
  category: 'Store Locator Widget',
  color: '#FF6B35',
  detect: {
    scripts: [/storepoint\\.co/i],
    html: [/storepoint-locator/i, /data-storepoint/i],
  }
}

```

```

},
{
  name: 'Stockist',
  category: 'Store Locator Widget',
  color: '#2ECC71',
  detect: {
    scripts: [/stockist\.co/i],
    html: [/stockist-widget/i, /data-stockist/i],
  }
},
{
  name: 'Bullseye Locations',
  category: 'Store Locator Widget',
  color: '#E74C3C',
  detect: {
    scripts: [/bullseyelocations\.com/i, /bullseye\.js/i],
    html: [/bullseye-locator/i],
  }
},
{
  name: 'Yext',
  category: 'Store Locator Widget',
  color: '#0F70E0',
  detect: {
    scripts: [/yext\.com/i, /yextpages/i],
    html: [/yext-locator/i, /data-yext/i],
  }
},
{
  name: 'Locally.io',
  category: 'Store Locator Widget',
  color: '#00B4D8',
  detect: {
    scripts: [/locally\.io/i, /locally-sdk/i],
    html: [/locally-widget/i],
  }
},
{
  name: 'Store Locator Plus (WordPress)',
  category: 'Store Locator Widget',
  color: '#21759B',
  detect: {
    scripts: [/store-locator-plus/i, /slp_core/i],
    html: [/slp_results/i, /store-locator-plus/i],
  }
},
{
  name: 'Shopify Store Locator (Secomapp)',

```

```

category: 'Store Locator Widget',
color: '#96BF48',
detect: {
  scripts: [/secomapp.*store.*locator/i, /storelocator\.secomapp/i],
  html: [/secomapp-storelocator/i],
}
},

```

Now, when you run TechScanner on a brand, you already know which widget they use. The store count pipeline can skip the generic Puppeteer scrape and go straight to the widget's API.

Dedicated Widget Parsers

Each parser is a function that takes the brand's domain and returns structured store data:

```

javascript
// server/widget-parsers.js

```

```

const axios = require('axios');

```

```

const WIDGET_PARSERS = {

```

```

  // ——— STOREMAPPER —————

```

```

  // Storemapper embeds contain a data attribute or script
  // with the account ID. Their API is public.

```

```

  'Storemapper': {
    async extractStoreId(page) {
      return await page.evaluate(() => {
        // Method 1: data attribute
        const el = document.querySelector('[data-storemapper-id]');
        if (el) return el.getAttribute('data-storemapper-id');

```

```

        // Method 2: script src contains ID
        const scripts = Array.from(document.querySelectorAll('script'));
        for (const s of scripts) {
          const match = s.src?.match(/storemapper\.coVjsV(\d+)/);
          if (match) return match[1];
        }

```

```

        // Method 3: inline script contains ID
        for (const s of scripts) {
          const match = s.textContent?.match(
            /storemapper.*?(d{4,})/
          );
          if (match) return match[1];
        }

```

```

        return null;
      });

```

```

},

async fetchStores(storeId) {
  const url = `https://www.storemapper.co/api/users/${storeId}/stores.json`;
  const { data } = await axios.get(url, { timeout: 10000 });

  return data.map(store => ({
    name: store.name,
    address: store.address,
    city: store.city,
    state: store.state,
    country: store.country,
    zip: store.postal,
    lat: parseFloat(store.latitude),
    lng: parseFloat(store.longitude),
    phone: store.phone,
    source: 'storemapper',
  }));
}
},

```

```

// ——— STOREPOINT —————
'Storepoint': {
  async extractStoreId(page) {
    return await page.evaluate(() => {
      const el = document.querySelector('[data-storepoint-id]');
      if (el) return el.getAttribute('data-storepoint-id');

      const scripts = Array.from(document.querySelectorAll('script'));
      for (const s of scripts) {
        const match = s.src?.match(/storepoint\.co\/api\/v1\/(\w+)/);
        if (match) return match[1];
        const textMatch = s.textContent?.match(
          /storepoint.*?id["'\s]+["']?\s?\w{10,}/
        );
        if (textMatch) return textMatch[1];
      }
      return null;
    });
  },

```

```

  async fetchStores(storeId) {
    const url = `https://api.storepoint.co/v1/${storeId}/locations`;
    const { data } = await axios.get(url, { timeout: 10000 });

    const locations = data?.results?.locations || [];
    return locations.map(loc => ({
      name: loc.name,

```

```

        address: loc.streetaddress,
        city: loc.city,
        state: loc.state,
        country: loc.country,
        zip: loc.postcode,
        lat: parseFloat(loc.loc_lat),
        lng: parseFloat(loc.loc_long),
        phone: loc.phone,
        tags: loc.tags,
        source: 'storepoint',
    }));
    }
},

// ——— STOCKIST —————
'Stockist': {
  async extractStoreId(page) {
    return await page.evaluate(() => {
      const el = document.querySelector('[data-stockist-widget-tag]');
      if (el) return el.getAttribute('data-stockist-widget-tag');

      const scripts = Array.from(document.querySelectorAll('script'));
      for (const s of scripts) {
        const match = s.src?.match(/stockist\.co\/embed\/v1\/widget\/(\w+)/);
        if (match) return match[1];
      }
      return null;
    });
  },

  async fetchStores(storeId) {
    const url = `https://stockist.co/api/v1/${storeId}/locations`;
    const { data } = await axios.get(url, { timeout: 10000 });

    return (data || []).map(loc => ({
      name: loc.name,
      address: [loc.address_line_1, loc.address_line_2]
        .filter(Boolean).join(', '),
      city: loc.city,
      state: loc.state,
      country: loc.country,
      zip: loc.postal_code,
      lat: parseFloat(loc.latitude),
      lng: parseFloat(loc.longitude),
      phone: loc.phone,
      source: 'stockist',
    }));
  }
}

```

```

},

// ——— YEXT —————
'Yext': {
  async extractConfig(page) {
    return await page.evaluate(() => {
      // Yext often injects config into window.__YEXT_CONFIG__
      // or into a data attribute
      if (window.__YEXT_CONFIG__) return window.__YEXT_CONFIG__;

      const el = document.querySelector('[data-yext-api-key]');
      if (el) return {
        apiKey: el.getAttribute('data-yext-api-key'),
        experienceKey: el.getAttribute('data-yext-experience-key'),
      };
      return null;
    });
  },

  async fetchStores(config) {
    // Yext's API requires the experience key and API key
    // This is trickier — may need to intercept their
    // liveapi.yext.com calls via XHR interceptor instead
    // Fall back to XHR intercept for Yext
    return null; // Signal to use generic interceptor
  }
},
};

```

```

// ——— ORCHESTRATOR —————
async function fetchStoresViaWidget(page, widgetName, storeLocatorUrl) {
  const parser = WIDGET_PARSERS[widgetName];
  if (!parser) return null;

  // Navigate to store locator page first
  await page.goto(storeLocatorUrl, {
    waitUntil: 'networkidle2',
    timeout: 30000
  });

  // Extract the widget's account/store ID
  const storeId = parser.extractStoreId
    ? await parser.extractStoreId(page)
    : await parser.extractConfig?.(page);

  if (!storeId) return null;

  // Fetch stores directly from widget API

```

```

const stores = await parser.fetchStores(storeId);
return stores;
}

module.exports = { WIDGET_PARSERS, fetchStoresViaWidget };

```

Widget Coverage Analysis

Here's my estimate of what percentage of D2C brands in India (your primary market) each widget covers. This should guide your build order:

Widget	Est. Coverage (India D2C)	Build Priority	Difficulty
Custom/proprietary store locator	~40%	Handle via generic XHR interceptor	Already built above
Storemapper	~12%	Build first	Easy — clean public API
Shopify native store locator apps	~15%	Build second	Medium — varies by app
Storepoint	~8%	Build third	Easy — clean public API
Stockist	~5%	Build fourth	Easy — clean public API
Google Maps embed only	~10%	Use Tier 2 (Places API)	N/A
Yext	~3%	XHR intercept	Medium
No store locator found	~7%	Tier 2 + manual	N/A

With just the generic XHR interceptor + Storemapper + Storepoint + Stockist parsers, you cover roughly 75% of brands with high accuracy. That's your v1 target.

3. Confidence Scoring Model — Production-Ready

The scoring model I outlined earlier was conceptual. Here's the full implementation with weighted factors and decay:

javascript

// server/store-confidence.js

```
function calculateStoreConfidence({
  source,          // 'widget_api' | 'xhr_intercept' | 'dom_parsing' |
                  // 'google_places' | 'manual' | 'news_inference'
  storeCount,
  previousCount,   // from last scrape (null if first time)
  previousScrapedAt, // Date of last scrape
  googlePlacesCount, // validation count from Google (null if not run)
  locatorPageExists, // boolean
  locatorLastModified, // Date from HTTP header (null if unavailable)
  hasFranchiseModel, // boolean (known from brand profile)
  hasStructuredData, // boolean (JSON-LD found)
  storesHaveCoords, // boolean (lat/lng present)
  storesHaveAddresses, // boolean
  manuallyVerifiedAt, // Date (null if never verified)
}) {
  let score = 0;
  const flags = [];

  // =====
  // SOURCE QUALITY (max 45 points)
  // =====

  const sourceScores = {
    'widget_api': 40,    // Best: clean structured data from known API
    'xhr_intercept': 35, // Very good: structured JSON from brand's API
    'manual': 35,        // Human verified
    'dom_parsing': 20,   // Decent but fragile
    'google_places': 10, // Noisy, over-counts
    'news_inference': 5, // Very approximate
  };

  score += sourceScores[source] || 0;

  // Bonus for structured coordinates
  if (storesHaveCoords) score += 5;

  // =====
  // CROSS-VALIDATION (max 20 points)
  // =====

  if (googlePlacesCount !== null && storeCount > 0) {
    const ratio = googlePlacesCount / storeCount;
```

```

if (ratio >= 0.7 && ratio <= 1.3) {
  // Within 30% — strong validation
  score += 20;
} else if (ratio >= 0.5 && ratio <= 2.0) {
  // Within 2x — moderate validation
  score += 10;
} else if (ratio > 3.0) {
  // Google shows 3x+ more — likely reseller contamination
  score += 5;
  flags.push('google_overcounts_significantly');
} else if (ratio < 0.3) {
  // Google shows much less — brand locator may include partners
  score += 0;
  flags.push('brand_locator_may_include_partners');
}
}

```

```

// =====
// TEMPORAL CONSISTENCY (max 15 points)
// =====

```

```

if (previousCount !== null && previousScrapedAt) {
  const daysSinceLast = (Date.now() - previousScrapedAt.getTime())
    / (1000 * 60 * 60 * 24);
  const changePercent = Math.abs(storeCount - previousCount)
    / Math.max(previousCount, 1) * 100;

  if (changePercent < 10) {
    // Stable count — high confidence
    score += 15;
  } else if (changePercent < 30) {
    // Moderate change — reasonable for growing brands
    score += 10;
  } else if (changePercent > 50 && daysSinceLast < 30) {
    // >50% change in under a month — suspicious
    score += 0;
    flags.push('suspicious_rapid_change');
  } else {
    score += 5;
  }
}

```

```

// =====
// DATA COMPLETENESS (max 10 points)
// =====

```

```

if (storesHaveAddresses) score += 5;

```

```

if (hasStructuredData) score += 3;
if (locatorPageExists) score += 2;

// =====
// MANUAL VERIFICATION BONUS (max 10 points, decays)
// =====

if (manuallyVerifiedAt) {
  const daysSinceVerification = (Date.now() - manuallyVerifiedAt.getTime())
    / (1000 * 60 * 60 * 24);

  if (daysSinceVerification < 30) score += 10;
  else if (daysSinceVerification < 90) score += 7;
  else if (daysSinceVerification < 180) score += 3;
  // After 6 months, manual verification adds nothing
}

// =====
// DEDUCTIONS
// =====

if (hasFranchiseModel) {
  score -= 10;
  flags.push('franchise_model');
}

if (storeCount === 0 && locatorPageExists) {
  score -= 15;
  flags.push('locator_exists_but_no_stores_found');
}

// =====
// NORMALIZE TO 0-100
// =====

const finalScore = Math.max(0, Math.min(100, score));

// Determine display tier
let tier;
if (finalScore >= 70) tier = 'high';
else if (finalScore >= 40) tier = 'approximate';
else tier = 'low';

return {
  score: finalScore,
  tier,
  flags,
  displayRecommendation: getDisplayRecommendation(tier, storeCount, flags),
}

```

```

    };
  }

function getDisplayRecommendation(tier, count, flags) {
  if (tier === 'high') {
    return {
      show: true,
      format: `${count} stores`,
      qualifier: null,
    };
  }

  if (tier === 'approximate') {
    // Round to nearest meaningful number
    const rounded = count < 20 ? count
      : count < 100 ? Math.round(count / 5) * 5
      : Math.round(count / 10) * 10;

    const label = flags.includes('franchise_model')
      ? `~${rounded} retail locations`
      : `~${rounded} stores`;

    return {
      show: true,
      format: label,
      qualifier: 'approximate',
    };
  }

  // Low confidence
  return {
    show: false,
    format: 'Store data unavailable',
    qualifier: 'insufficient_data',
  };
}

module.exports = { calculateStoreConfidence };

```

How This Integrates Into the UI

The confidence tier drives what your users see on account cards and detail pages:

High confidence (70+): Display as "42 stores" with a small green check icon. No qualifier needed. This is what Raj and Priya trust to make outreach decisions.

Approximate (40-69): Display as "~40 stores" in a slightly muted style with an "approximate" tooltip. When clicked, show: "Based on [source]. Last verified [date]." Good enough for filtering and prioritization, but users know not to quote it in a pitch email.

Low confidence (<40): Don't show a number at all. Display "Offline presence detected" if you know stores exist but can't count them, or simply omit the field. This is critical — never show a number you don't trust. Vikram the VC will lose faith in your entire dataset if he catches one bad store count.

The Flags System

Flags serve two purposes. First, they help your data ops team prioritize manual review. Run a weekly query: "Show me all brands where flags contains suspicious_rapid_change or locator_exists_but_no_stores_found." Those are your highest-value manual QA targets. Second, flags feed into the signal detection system for the Market Intelligence dashboard. When a brand's store count jumps by >30% between scrapes and confidence is high, that's an "Omnichannel Expansion" signal. When it drops by >20%, that's a potential retrenchment signal worth alerting watchlist subscribers about.

Full Pipeline Orchestration

Here's how all three pieces fit together in the complete store detection flow:

javascript

// server/store-pipeline.js

```
async function detectStores(brand) {
  const { domain, techStack, storeLocatorUrl } = brand;

  // Step 1: Check if we already know the widget type
  // (from TechScanner data)
  const widgetTech = techStack?.find(t =>
    t.category === 'Store Locator Widget'
  );

  let stores = null;
  let source = null;

  // Step 2: Try widget-specific parser first (fastest + most reliable)
  if (widgetTech && storeLocatorUrl) {
    const page = await getBrowserPage();
    try {
      stores = await fetchStoresViaWidget(
        page, widgetTech.name, storeLocatorUrl
      );
      if (stores?.length > 0) source = 'widget_api';
    } finally {
      await page.close();
    }
  }

  // Step 3: If no widget or widget parser failed,
```

```

// try generic XHR interception
if (!stores || stores.length === 0) {
  const locatorUrl = storeLocatorUrl
  || await discoverStoreLocatorUrl(domain);

  if (locatorUrl) {
    const page = await getBrowserPage();
    try {
      const captured = await interceptStoreAPIs(page, locatorUrl);

      if (captured.length > 0) {
        // Pick the highest-scored response
        const best = captured.sort((a, b) => b.score - a.score)[0];
        stores = parseGenericStoreResponse(best.data);
        source = 'xhr_intercept';
      }

      // Step 4: If XHR found nothing, try DOM fallback
      if (!stores || stores.length === 0) {
        stores = await fallbackDOMParsing(page);
        if (stores?.length > 0) source = 'dom_parsing';
      }
    } finally {
      await page.close();
    }
  }
}

// Step 5: Deduplicate and normalize
if (stores && stores.length > 0) {
  stores = deduplicateStores(stores);
}

// Step 6: Optional — Google Places validation
// Only run for brands on active watchlists (cost control)
let googlePlacesCount = null;
if (brand.isOnActiveWatchlist) {
  googlePlacesCount = await getGooglePlacesCount(
    brand.name, brand.country
  );
}

// Step 7: Calculate confidence
const previous = await getLastStoreRecord(domain);

const confidence = calculateStoreConfidence({
  source: source || 'none',
  storeCount: stores?.length || 0,

```

```

previousCount: previous?.current_count || null,
previousScrapedAt: previous?.last_scraped || null,
googlePlacesCount,
locatorPageExists: !!storeLocatorUrl,
hasFranchiseModel: brand.hasFranchiseModel || false,
hasStructuredData: stores?.some(s => s.source === 'json_ld'),
storesHaveCoords: stores?.some(s => s.lat && s.lng),
storesHaveAddresses: stores?.some(s => s.address || s.city),
manuallyVerifiedAt: previous?.manually_verified_at || null,
});

```

// Step 8: Save to database

```

const record = {
  domain,
  current_count: stores?.length || 0,
  previous_count: previous?.current_count || null,
  confidence_score: confidence.score,
  confidence_tier: confidence.tier,
  confidence_flags: confidence.flags,
  display: confidence.displayRecommendation,
  source,
  store_locator_url: storeLocatorUrl,
  last_scraped: new Date(),
  last_changed: (stores?.length !== previous?.current_count)
    ? new Date()
    : previous?.last_changed,
  stores: stores || [],
};

```

```

await upsertStoreRecord(domain, record);

```

```

return record;

```

```

}

```

Three Risks to Watch

Risk 1 — Widget APIs changing or requiring auth. Storemapper's public API could add rate limiting or auth tomorrow. Mitigation: always have the generic XHR interceptor as fallback.

Design the pipeline so widget parsers are an optimization, not a dependency.

Risk 2 — Over-investing in edge cases. The 40% of brands with fully custom store locators will eat your engineering time. Don't try to build a perfect parser for every custom implementation. The XHR interceptor handles ~70% of custom cases. For the rest, flag as "low confidence" and route to manual data ops. Your data ops person's time is cheaper than building and maintaining 50 custom scrapers.

Risk 3 — Legal/ToS considerations. Scraping store locator pages and hitting widget APIs at scale could trigger rate limits or ToS concerns. Mitigation: respectful scraping (add delays between requests, respect robots.txt for the brand's own pages), cache aggressively (don't

re-scrape a brand more than once per week unless a change signal triggers it), and for widget APIs, you're accessing public endpoints that the widgets themselves call from the browser — functionally equivalent to a user visiting the page